

EXPERIMENT 7

SLIDING WINDOW PROTOCOL

Flow control at the Data Link Layer using a sliding-window protocol

Objective: Implement a program to simulate the flow of frames from a sender to a receiver using a configurable window size, acknowledgements, and retransmission.

1. AIM

To write a program to implement flow control at the data link layer using the sliding window protocol and simulate the flow of frames from one node to another.

The program accepts the window size and a text message from the user. Each character is treated as one frame. Frames are sent from the sender to the receiver, acknowledgements are checked, and frames are retransmitted when a NACK or incorrect acknowledgement is received.

Key requirements

- Accept a window size from the user.
- Convert every character of the message into a frame.
- Send frames according to the window size.
- Process ACK/NACK responses.
- Retransmit the required frame when an error is detected.

2. THEORY – SLIDING WINDOW PROTOCOL

The sliding window protocol is a flow-control mechanism used at the data link layer to control the transmission of frames between a sender and receiver.

Instead of sending only one frame and waiting for its acknowledgement, the sender can transmit multiple frames before waiting for acknowledgements. The number of frames that can be sent at a time is called the window size.

After receiving acknowledgements for frames, the transmission range moves forward and the sender can transmit the next set of frames. This movement of the transmission range is called sliding the window.

Acknowledgement and error handling

After receiving a frame, the receiver checks whether the frame number is the expected one. The sender uses the acknowledgement to decide which frames should be sent next. If the frame number is incorrect, the receiver generates a NACK and the required frame is retransmitted.

3. SENDER ALGORITHM

1. Start the program.
2. Get the window size from the user.
3. Get the text message from the user.
4. Convert every character into one frame.
5. Assign a frame number to every character.
6. Send frames according to the window size.
7. Store the transmitted frames in the sender buffer.
8. Introduce a delay to simulate transmission.
9. Read the receiver buffer.
10. Check the received ACK/NACK.
11. If ACK is received, slide the window and send the next set of frames.
12. If NACK is received, retransmit the required frame.
13. Continue until all frames are successfully transmitted.
14. Stop.

4. RECEIVER ALGORITHM

1. Start the receiver.
2. Read the sender buffer.
3. Read the frame numbers.
4. Check whether the frame number is the expected number.
5. If the frame number is correct, accept the frame.
6. Generate the appropriate ACK.
7. If the frame number is incorrect, generate a NACK.
8. Continue until all frames are received.

Error simulation

The handwritten procedure specifies that the user can manually modify the frame number in sender_buffer or ACK/NACK information in receiver_buffer to verify protocol behaviour.

5. PROGRAM – SENDER / RECEIVER

Frame creation and buffer handling

```
import time
import os

SENDER_FILE = "Sender_Buffer.txt"
RECEIVER_FILE = "Receiver_Buffer.txt"

def create_frames(message):
    frames = []
    for i, ch in enumerate(message):
        frames.append({"frame_no": i, "data": ch})
    return frames

def write_sender_buffer(frames):
    with open(SENDER_FILE, "w") as file:
        for frame in frames:
            file.write(
                f"Frame {frame['frame_no']} Data {frame['data']}\n"
            )

def write_receiver_response(ack_type, ack_no):
    with open(RECEIVER_FILE, "w") as file:
        file.write(f"{ack_type} {ack_no}\n")

def read_ack():
    if not os.path.exists(RECEIVER_FILE):
        return None, None
    with open(RECEIVER_FILE, "r") as file:
        line = file.readline().strip().split()
    if len(line) < 2:
        return None, None
    return line[0], int(line[1])
```

5. PROGRAM – RECEIVER LOGIC

```
def receiver(frames, expected):
    if expected >= len(frames):
        return "ACK", expected

    frame_no = frames[expected]["frame_no"]

    # Receiver checks whether the frame is expected.
    if frame_no == expected:
        write_receiver_response("ACK", frame_no)
        print(f"ACK {frame_no} received")
        return "ACK", frame_no

    write_receiver_response("NACK", expected)
    print(f"NACK {expected} received")
    return "NACK", expected
```

Main sliding-window transmission

```
window_size = int(input("Enter window size: "))
message = input("Enter text message: ")

frames = create_frames(message)
write_sender_buffer(frames)

base = 0
while base < len(frames):
    end = min(base + window_size, len(frames))

    print("\nSending frames:", list(range(base, end)))
    time.sleep(1)

    # Simulate receiver acknowledgement.
    ack_type, ack_no = receiver(frames, base)

    if ack_type == "ACK":
        base = ack_no + 1
    elif ack_type == "NACK":
        print("Retransmitting frame...")
        time.sleep(2)
print("\nAll frames transmitted successfully.")
```

```
print("Transmission completed.")
```

6. INPUT AND OUTPUT

Sample Input

Enter window size: 3

Enter text message: HELLO

Frame representation

Frame No.	Data	Window
0	H	1st
1	E	2nd
2	L	3rd
3	L	Next after ACK
4	O	Next after ACK

Sample Output

Initial transmission:

ACK 2 received

Next window is transmitted after the acknowledgement.

All frames transmitted successfully.

Transmission completed.

7. OBSERVATION

The sliding window protocol was used to control the transmission of multiple frames between sender and receiver. The window size determines how many frames can be transmitted before the sender processes acknowledgements.

ACK indicates successful reception of a frame. NACK indicates that the expected frame was not received correctly and causes the required frame to be retransmitted.

Challenges faced

- Handling the sender and receiver buffers.
- Maintaining the current window/base position.
- Checking ACK and NACK values.
- Simulating retransmission after an error.
- Understanding how the window moves after acknowledgement.

8. RESULT

Thus, the sliding window protocol for flow control at the data link layer was successfully implemented and executed. Frames were generated from the input message, transmitted according to the selected window size, acknowledged by the receiver, and retransmitted when required.

CONCLUSION

The experiment demonstrates how sliding-window flow control improves frame transmission by allowing multiple frames to be in transit while using acknowledgements to control and verify delivery.